COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	October 2, 2026	**DATE SUBMITTED:**	October 2, 2026
NAME:	Clifford Lee C. Doplayna Almirol Joel A	**INSTRUCTOR:**	Engr. Jimlord Quejado

Hands-on Activity 10.1
Graphs

1. PROCEDURE OUTPUT

ILO A: Create C++ code for graph implementation utilizing adjacency matrix and adjacency list

Adjacency.cpp > DiaGraph

```cpp
#include <iostream>

// stores adjacency list items
struct adjNode {
    int val, cost;
    adjNode* next;
};

// structure to store edges
struct graphEdge {
    int start_ver, end_ver, weight;
};

class DiaGraph {
    // insert new nodes into adjacency list from given graph
    adjNode* getAdjListNode(int value, int weight, adjNode* head) {
        adjNode* newNode = new adjNode;
        newNode->val = value;
        newNode->cost = weight;
        newNode->next = head; // point new node to current head
        return newNode;
    }

    int N; // number of nodes in the graph

public:
    adjNode** head; // adjacency list as array of pointers

    // Constructor
    DiaGraph(graphEdge edges[], int n, int N) {
        // allocate new node
        head = new adjNode*[N]();
        this->N = N;

        // initialize head pointer for all vertices
        for (int i = 0; i < N; ++i)
            head[i] = nullptr;

        // construct directed graph by adding edges to it
        for (unsigned i = 0; i < n; i++) {
            int start_ver = edges[i].start_ver;
            int end_ver = edges[i].end_ver;
            int weight = edges[i].weight;

            // insert in the beginning
            adjNode* newNode =
                getAdjListNode(end_ver, weight, head[start_ver]);
```

```
49              // point head pointer to new node
50              head[start_ver] = newNode;
51          }
52      }
53
54      // Destructor
55      ~DiaGraph() {
56          for (int i = 0; i < N; i++) {
57              adjNode* current = head[i];
58
59              while (current != nullptr) {
60                  adjNode* t;
61                  current = t;
62                  delete temp;
63              }
64          }
65
66          delete[] head;
67      }
68  };
69
70  // print all adjacent vertices of given vertex
71  void display_AdjList(adjNode* ptr, int i) {
72      while (ptr != nullptr) {
73          std::cout << "(" << i << ", " << ptr->val
74                    << ", " << ptr->cost << ") ";
75          ptr = ptr->next;
76      }
77
78      std::cout << std::endl;
79  }
80
81  // graph implementation
82  int main() {
83      // graph edges array
84      graphEdge edges[] = {
85          // (x, y, w) -> edge from x to y with weight w
86          {0, 1, 2},
87          {0, 2, 4},
88          {1, 4, 3},
89          {2, 3, 2},
90          {3, 1, 4},
```

adjNode *temp

```
        {4, 3, 3}
    };

    int N = 6; // Number of vertices in the graph

    // calculate number of edges
    int n = sizeof(edges) / sizeof(edges[0]);

    // construct graph
    DiaGraph diagraph(edges, n, N);

    // print adjacency list representation of graph
    std::cout << "Graph adjacency list "
              << std::endl
              << "(start_vertex, end_vertex, weight):"
              << std::endl;

    for (int i = 0; i < N; i++) {
        // display adjacent vertices of vertex i
        display_AdjList(diagraph.head[i], i);
    }

    return 0;
}

```

Analysis:
A.1, A.2, and A.3 focus on the basic implementation of a graph. A.1 (Create a Graph) defines the structures needed to represent graph vertices and edges, including the starting vertex, ending vertex, and edge weight. A.2 (Adjacency Matrix) introduces the matrix representation of a graph. A.3 (Adjacency List) represents the graph using linked nodes, where each vertex stores its connected vertices and their corresponding edge costs.

A.1. Create a Graph
A.2. Adjacency Matrix
A.3. Adjacency List

ILO B: Create C++ code for implementing graph traversal algorithms such as Breadth-First and Depth-First Search

DFS.cpp > ...

```cpp
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <stack>

template <typename T>
class Graph;

template <typename T>
struct Edge {
    size_t src;
    size_t dest;
    T weight;

    inline bool operator<(const Edge<T> &e) const {
        return this->weight < e.weight;
    }

    inline bool operator>(const Edge<T> &e) const {
        return this->weight > e.weight;
    }
};

template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G) {
    for (auto i = 1; i <= G.vertices(); i++) {
        os << i << ":\t";

        auto edges = G.outgoing_edges(i);

        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";

        os << std::endl;
    }

    return os;
}

template <typename T>
class Graph {
public:

    Graph(size_t N) : V(N) {
    }
```

```cpp

    auto vertices() const {
        return V;
    }

    auto &edges() const {
        return edge_list;
    }

    void add_edge(Edge<T> &&e) {
        if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    }

    auto outgoing_edges(size_t v) const {
        std::vector<Edge<T>> edges_from_v;

        for (auto &e : edge_list) {
            if (e.src == v)
                edges_from_v.emplace_back(e);
        }

        return edges_from_v;
    }

    template <typename U>
    friend std::ostream &operator<<(std::ostream &os,
                                    const Graph<U> &G);

private:
    size_t V;
    std::vector<Edge<T>> edge_list;
};

template <typename T>
auto depth_first_search(const Graph<T> &G, size_t dest) {
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;

```

```
91      stack.push(1);
92
93      while (!stack.empty()) {
94          auto current_vertex = stack.top();
95          stack.pop();
96
97          if (visited.find(current_vertex) == visited.end()) {
98              visited.insert(current_vertex);
99              visit_order.push_back(current_vertex);
100
101             for (auto e : G.outgoing_edges(current_vertex)) {
102                 if (visited.find(e.dest) == visited.end()) {
103                     stack.push(e.dest);
104                 }
105             }
106         }
107     }
108
109     return visit_order;
110 }
111
112 template <typename T>
113 auto create_reference_graph() {
114     Graph<T> G(9);
115
116     std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
117
118     edges[1] = {{2, 0}, {5, 0}};
119     edges[2] = {{1, 0}, {5, 0}, {4, 0}};
120     edges[3] = {{4, 0}, {7, 0}};
121     edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
122     edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
123     edges[6] = {{4, 0}, {7, 0}, {8, 0}};
124     edges[7] = {{3, 0}, {6, 0}};
125     edges[8] = {{4, 0}, {5, 0}, {6, 0}};
126
127     for (auto &i : edges)
128         for (auto &j : i.second)
129             G.add_edge(Edge<T>{i.first, j.first, j.second});
130
131     return G;
132 }
133
134 template <typename T>
135 void test_DFS() {
```

```cpp
    auto G = create_reference_graph<unsigned>();

    std::cout << G << std::endl;

    std::cout << "DFS Order of vertices: " << std::endl;

    auto dfs_visit_order = depth_first_search(G, 1);

    for (auto v : dfs_visit_order)
        std::cout << v << std::endl;
}

int main() {
    using T = unsigned;

    test_DFS<T>();

    return 0;
}

```

TERMINAL

```
PS C:\Users\TIPQC\OneDrive\Documents\HOAGRAPH> & C:\Users\TIPQC\OneDrive\Documents\HOAGRAPH\DFS.exe
1:      {2: 0}, {5: 0},
2:      {1: 0}, {5: 0}, {4: 0},
3:      {4: 0}, {7: 0},
4:      {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0},
5:      {1: 0}, {2: 0}, {4: 0}, {8: 0},
6:      {4: 0}, {7: 0}, {8: 0},
7:      {3: 0}, {6: 0},
8:      {4: 0}, {5: 0}, {6: 0},
9:

DFS Order of vertices:
1
5
8
6
7
3
4
2
PS C:\Users\TIPQC\OneDrive\Documents\HOAGRAPH>
```

Analysis:
The DFS code uses a stack to visit the vertices of the graph. It starts at vertex 1, visits an unvisited vertex and continues going into other graphs before going back to another vertex. The visited set prevents the same vertex from being visited again.

B.1. Depth-First Search

BFS.cpp > ...

```cpp
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <queue>

template <typename T>
class Graph;

template <typename T>
struct Edge {
    size_t src;
    size_t dest;
    T weight;

    inline bool operator<(const Edge<T> &e) const {
        return this->weight < e.weight;
    }

    inline bool operator>(const Edge<T> &e) const {
        return this->weight > e.weight;
    }
};

template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G) {
    for (auto i = 1; i <= G.vertices(); i++) {
        os << i << ":\t";

        auto edges = G.outgoing_edges(i);

        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";

        os << std::endl;
    }

    return os;
}

template <typename T>
class Graph {
public:

    Graph(size_t N) : V(N) {}

    auto vertices() const {
```

```
49          return V;
50      }
51
52      auto &edges() const {
53          return edge_list;
54      }
55
56      void add_edge(Edge<T> &&e) {
57          if (e.src >= 1 && e.src <= V &&
58              e.dest >= 1 && e.dest <= V)
59              edge_list.emplace_back(e);
60          else
61              std::cerr << "Vertex out of bounds" << std::endl;
62      }
63
64      auto outgoing_edges(size_t v) const {
65          std::vector<Edge<T>> edges_from_v;
66
67          for (auto &e : edge_list) {
68              if (e.src == v) {
69                  edges_from_v.emplace_back(e);
70              }
71          }
72
73          return edges_from_v;
74      }
75
76      template <typename U>
77      friend std::ostream &operator<<(std::ostream &os,
78                                      const Graph<U> &G);
79
80  private:
81      size_t V;
82      std::vector<Edge<T>> edge_list;
83  };
84
85  template <typename T>
86  auto create_reference_graph() {
87      Graph<T> G(9);
88
89      std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
90
91      edges[1] = {{2, 2}, {5, 3}};
92      edges[2] = {{1, 2}, {5, 5}, {4, 1}};
93      edges[3] = {{4, 2}, {7, 3}};
```

```cpp
    edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
    edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
    edges[6] = {{4, 4}, {7, 4}, {8, 1}};
    edges[7] = {{3, 3}, {6, 4}};
    edges[8] = {{4, 5}, {5, 3}, {6, 1}};

    for (auto &i : edges)
        for (auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});

    return G;
}

template <typename T>
auto breadth_first_search(const Graph<T> &G, size_t dest) {
    std::queue<size_t> queue;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;

    queue.push(1);

    while (!queue.empty()) {
        auto current_vertex = queue.front();
        queue.pop();

        if (visited.find(current_vertex) == visited.end()) {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);

            for (auto e : G.outgoing_edges(current_vertex)) {
                if (visited.find(e.dest) == visited.end()) {
                    queue.push(e.dest);
                }
            }
        }
    }

    return visit_order;
}

template <typename T>
void test_BFS() {
    auto G = create_reference_graph<unsigned>();

    std::cout << G << std::endl;
```

```
139
140         std::cout << "BFS Order of vertices: " << std::endl;
141
142         auto bfs_visit_order = breadth_first_search(G, 1);
143
144         for (auto v : bfs_visit_order)
145             std::cout << v << std::endl;
146     }
147
148     int main() {
149         using T = unsigned;
150
151         test_BFS<T>();
152
153         return 0;
154     }
155
```

```
TERMINAL
PS C:\Users\TIPQC\OneDrive\Documents\HOAGRAPH> & C:\Users\TIPQC\OneDrive\Documents\HOAGRAPH\BFS.exe
1:      {2: 2}, {5: 3},
2:      {1: 2}, {5: 5}, {4: 1},
3:      {4: 2}, {7: 3},
4:      {2: 1}, {3: 2}, {5: 2}, {6: 4}, {8: 5},
5:      {1: 3}, {2: 5}, {4: 2}, {8: 3},
6:      {4: 4}, {7: 4}, {8: 1},
7:      {3: 3}, {6: 4},
8:      {4: 5}, {5: 3}, {6: 1},
9:

BFS Order of vertices:
1
2
5
4
8
3
6
7
PS C:\Users\TIPQC\OneDrive\Documents\HOAGRAPH>
```

Analysis:
The BFS code uses a queue to visit the vertices of the graph. It starts at vertex 1 and visits all the nearby vertices first before moving to the next vertices. The visited set keeps track of the vertices that have already been visited to avoid repeated visits.

B.2. Breadth-First Search

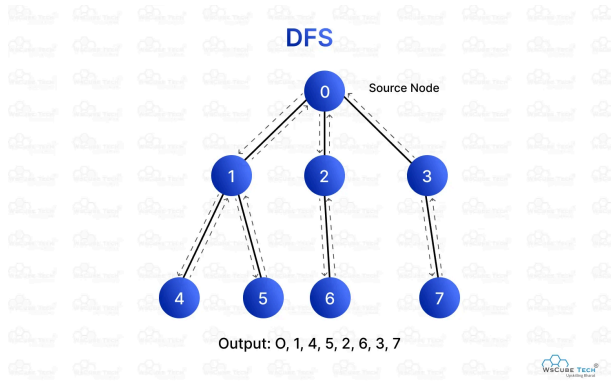
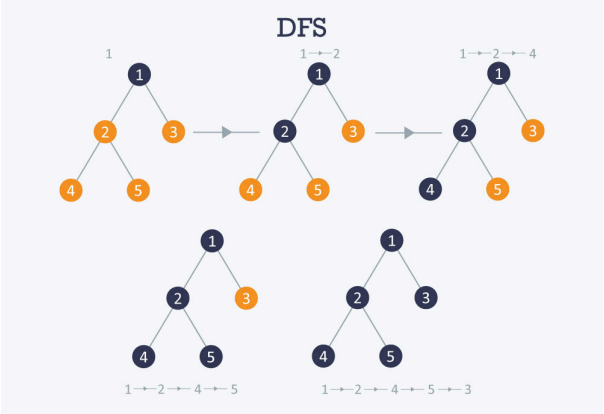
2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Demonstrate an understanding of graph implementation, operations and traversal methods

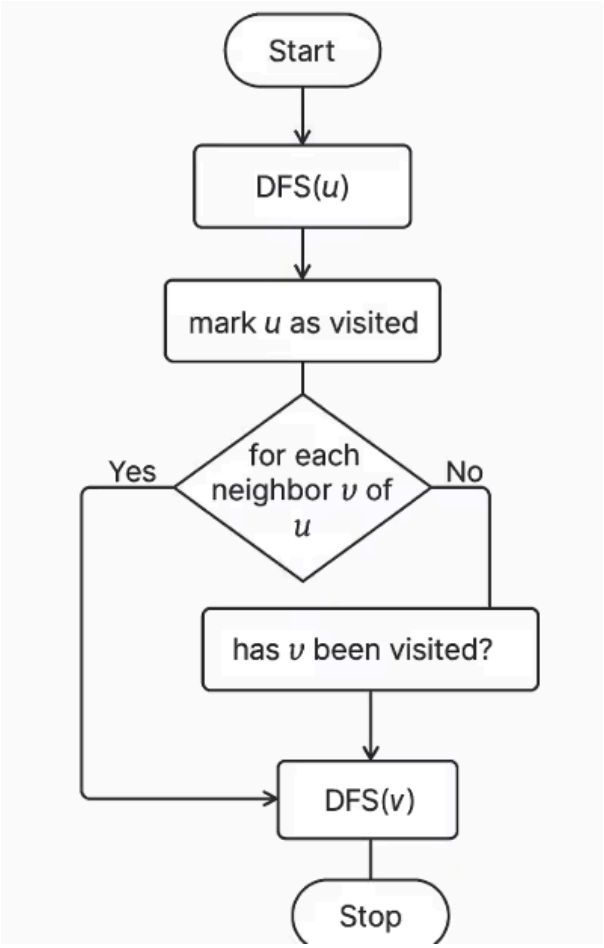
1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from the same vertex. Discuss which algorithm would be most helpful to accomplish this task.

The most appropriate algorithm for the given situation would be the Depth-First Search (DFS) algorithm because it goes as deep as possible along each path before returning. The DFS algorithm starts with an initial node and goes on to visit an unvisited adjacent node repeatedly until there is no node left to visit. At this point, the DFS algorithm returns to the node visited last in order to continue with other paths that were not yet explored.

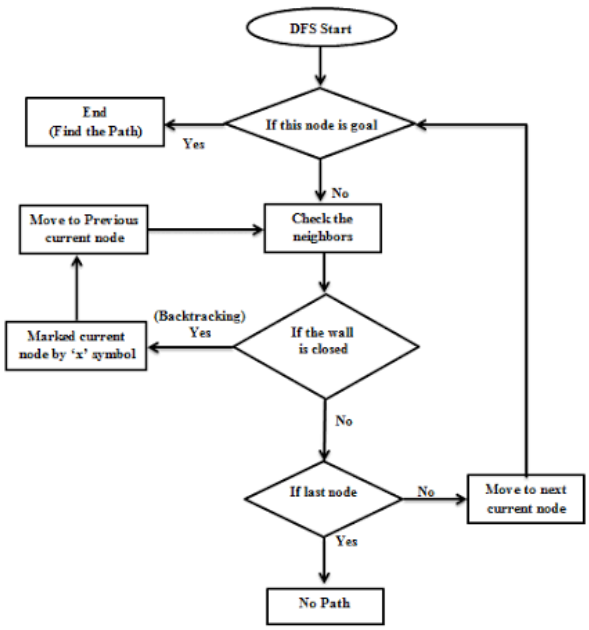
2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.

DFS in traversal strategies for trees	DFS in traversal strategies for trees
 www.wscubetech.com/resources/dsa/dfs-vs-bfs	 https://www.puppygraph.com/blog/dfs-graph

examine pseudocode	examine pseudocode



https://www.upgrad.com/blog/what-is-dfs-algorithm/



https://www.researchgate.net/figure/The-Flowchart-for-Depth-First-Search-Algorithm_fig6_358190357

3. In the performed code, what data structure is used to implement the Breadth First Search?

BFS works on a level-by-level basis (traversing all immediate neighbor vertices of a vertex before moving on to the next level). A Queue is utilized here to facilitate this, as it ensures that vertices are accessed in precisely the same order they are discovered.

4. How many times can a node be visited in the BFS?

In a typical BFS algorithm run on a graph, a boolean visited array/set is used to track nodes which have been discovered/enqueued already. Each node is visited/processed only once to avoid infinite loops (in case of cycles in graphs) and also for efficient time complexity (O(V+E)).

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Quillbot**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar checker**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **We used quillbot only for the grammar checker of our code analysis and also conclusion.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.

Conclusion

Doplayna:
After finishing this activity it helped me understand how graphs are implemented in this program using adjacency lists, as well as how DFS and BFS are used for graph traversal. I was able to follow the procedures and understand the codes, but the things that I still need to improve are my skills in writing and debugging graph programs.

T.I.P TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES
COMPUTER ENGINEERING
QUEZON CITY

Almirol:
By doing this task, I have learned about the representation of graphs by adjacency matrix and adjacency list in C++. The use of the BFS algorithm helped me learn how to explore nodes by level using the concept of queues. Similarly, DFS algorithms helped me understand how to explore graphs deeply using the stack data structure. For improvement, I will work on writing my own graph algorithms and optimize graph **traversals for realistic scenarios.**

Rubrics

Rubric for SO 7 (2)

Criteria	Ratings						Pts
SO 7 PI 1 IILO4 Utilize lifelong learning skills in pursuit of personal development and excellence in professional practice. threshold: 4.8 pts	6 pts Excellent | Educational interests and pursuits exist and flourish outside classroom requirements,knowledge and/or experiences are pursued independently and applies knowledge learned into practice	5 pts Good | Educational interests and pursuits exist and flourish outside classroom requirements,knowledge and/or experiences are pursued independently	4 pts Satisfactory | Look beyond classroom requirements, showing interest in pursuing knowledge independently	3 pts Unsatisfactory | Begins to look beyond classroom requirements, showing interest in pursuing knowledge independently	2 pts Poor | Relies on classroom instruction only	1 pts Very Poor | No initiative or interest in acquiring new knowledge	6 pts
SO 7 PI 2 IILO4 Utilize lifelong learning skills in pursuit of personal development and excellence in professional practice. threshold: 4.8 pts	6 pts Excellent | Completes an assigned task independently and practices continuous improvement	5 pts Good | Completes an assigned task without supervision or guidance	4 pts Satisfactory | Requires minimal guidance to complete an assigned task	3 pts Unsatisfactory | Requires detailed or step-by-step instructions to complete a task	2 pts Poor | Shows little interest to complete a task independently	1 pts Very Poor | No interest to complete a task independently	6 pts
SO 7 PI 3 IILO4 Utilize lifelong learning skills in pursuit of personal development and excellence in professional practice. threshold: 4.8 pts	6 pts Excellent | Synthesizes and integrates information from a variety of sources; formulates a clear and precise perspective; draws appropriate conclusions	5 pts Good | Evaluate information from a variety of sources; formulates a clear and precise perspective.	4 pts Satisfactory | Analyze information from a variety of sources; formulates a clear and precise perspective.	3 pts Unsatisfactory | Apply the gathered information to formulate the problem	2 pts Poor | Gather and summarized the information from a variety of sources but failed to formulate the problem	1 pts Very Poor | Gather information from a variety of sources	6 pts
SO 7 PI 4 IILO4 Utilize lifelong learning skills in pursuit of personal development and excellence in professional practice. threshold: 4.8 pts	6 pts Excellent | Ideas are combined in original and creative ways in line with the new and emerging technology trends to solve a problem or address an issue.	5 pts Good | Ideas are creative and adapt the new knowledge to solve a problem or address an issue	4 pts Satisfactory | Ideas are creative in solving a problem, or address an issue	3 pts Unsatisfactory | Shows some creative ways to solve the problem	2 pts Poor | Shows initiative and attempt to develop creative ideas to solve the problem	1 pts Very Poor | Ideas are copied or restated from the sources consulted	6 pts

Total Points: 24